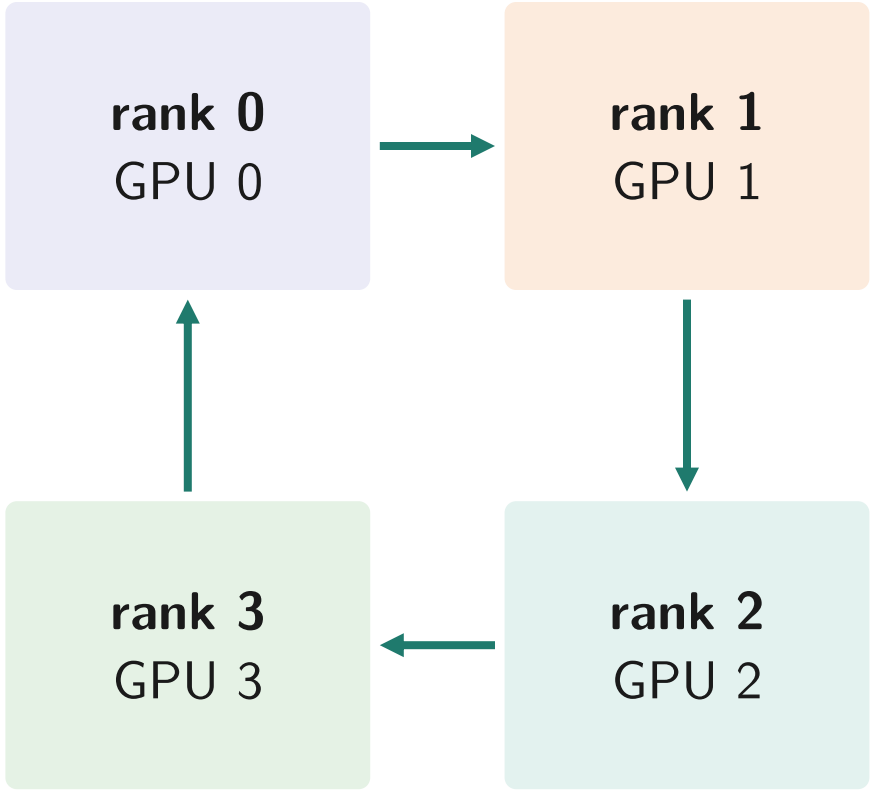
# Distributed Programming Foundations

Processes, ranks, messages, and collectives — and a two-number model of what they cost

Yuke Wang

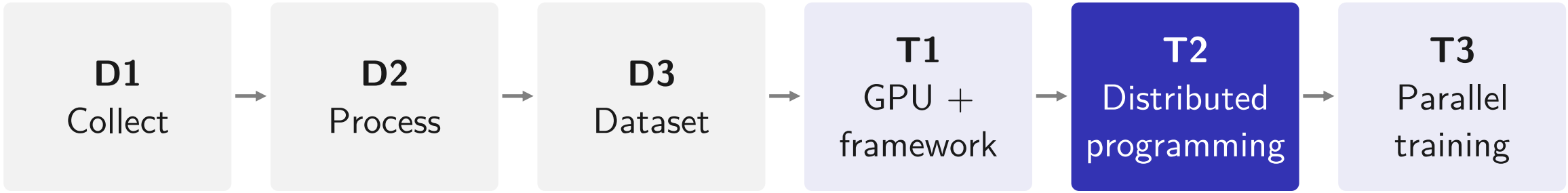
Rice University

COMP 329 · Spring 2027 · Week 10, Lecture 2




Four ranks, one GPU each, in a ring

# Where we are in the pipeline




T1 due Friday · T2 and Lab 4 released after this lecture

**In our pipeline**

T2: a CPU all-reduce from mpi4py messages, an NCCL benchmark, a two-server case, and an $\alpha$–$\beta$ **fit**. Today: the foundations of all four.

Distributed block: foundations (today) $\rightarrow$ NCCL (Week 11 L1) $\rightarrow$ multiple servers (Week 11 L2)

# Learning goals for today

- Explain why training needs **many processes**, with memory and time estimates.
- Use **world size, rank, and local rank**, and map each rank to its own GPU.
- Send and receive messages; explain **blocking**, **matching**, and **deadlock**.
- State the exact semantics of the **seven collectives**; build all-reduce from two of them.
- Fit the $\alpha$–$\beta$ **model** $T(n) = \alpha + n/\beta$ by least squares and **predict** a message time.

# A puzzle to start

**Option A**
1,000 messages of 1 KB

**Option B**
1 message of 1 MB

**Each message costs 1 µs to start, then moves 10 GB/s. Which option is faster, and by how much?**

**Key idea**

A: $1000 \times (1 + 0.1)$ µs $=$ **1.1 ms**. B: $1 + 100$ µs $\approx$ **0.1 ms**. Same bytes, **11×** apart: start-up cost dominates small messages.

Part 1

# Many processes, one program

Why more than one GPU, and how does each process know its job?

# Why one GPU is not enough: memory

$$7 \times 10^9 \times 16 \text{ bytes} = 112 \text{ GB}$$

one GPU: 80 GB

**Watch out**

Weights, gradients, and Adam state alone exceed one GPU. **Split them** across processes.




Source: Kaplan et al., "Scaling Laws for Neural Language Models," 2020 ($C \approx 6ND$); ZeRO, SC 2020 (16 B/param). H100 datasheet.

# Why one GPU is not enough: time

$$C \approx 6 \cdot N \cdot D = 6 \times 7 \times 10^9 \times 10^{12} = 4.2 \times 10^{22} \text{ FLOPs}$$

$$\text{one GPU at } 40\% \text{ of } 989 \text{ TFLOP/s} \approx 4.0 \times 10^{14} \text{ FLOP/s}$$

$$4.2 \times 10^{22} \,/\, 4.0 \times 10^{14} \approx 1.1 \times 10^8 \text{ seconds} \approx 3.4 \text{ years}$$

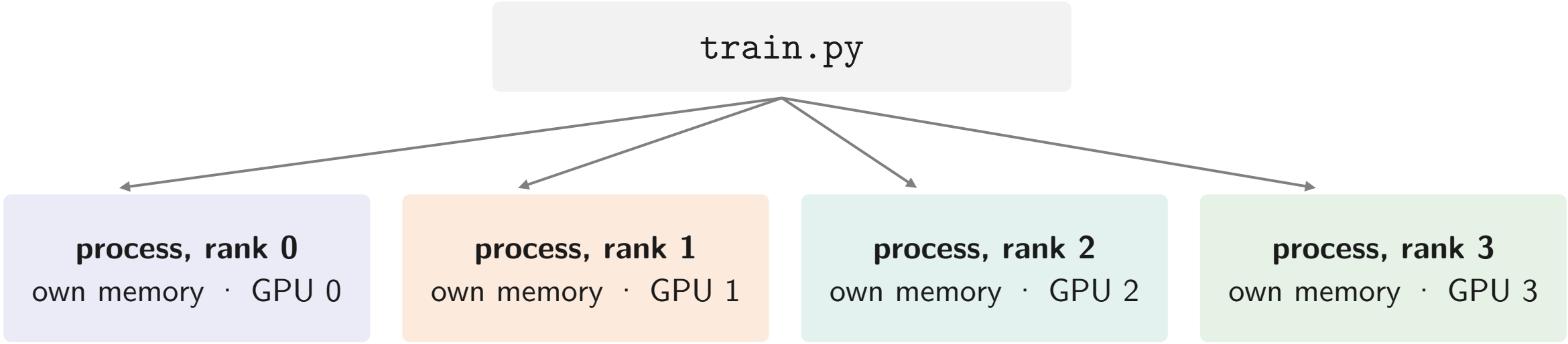
$$\text{1,024 GPUs, perfect scaling:} \quad \approx 1.2 \text{ days}$$

**In our pipeline**

"Perfect scaling" needs the GPUs to **communicate** cheaply every step — today's subject.

Source: Kaplan et al., 2020 ($C \approx 6ND$); H100 datasheet (BF16, dense); 40% utilization assumed.

# SPMD: one program, many processes




```
rank, p = comm.Get_rank(), comm.Get_size()
shard = dataset[rank::p]      # my share of the data
if rank == 0:
    log("only one process writes logs")
```

Same code everywhere; behaviour differs only through **rank**.

**No shared memory: processes cooperate only by messages.**

# World size, rank, and local rank

| node 0 | | | |
|---|---|---|---|
| GPU 0 | GPU 1 | GPU 2 | GPU 3 |
| **rank 0** local 0 | **rank 1** local 1 | **rank 2** local 2 | **rank 3** local 3 |

| node 1 | | | |
|---|---|---|---|
| GPU 0 | GPU 1 | GPU 2 | GPU 3 |
| **rank 4** local 0 | **rank 5** local 1 | **rank 6** local 2 | **rank 7** local 3 |

$$\text{rank} = \text{node} \times 4 + \text{local_rank}, \qquad \text{world size } p = 2 \times 4 = 8$$

**Key idea**

**rank** picks the work; **local rank** picks the GPU on this machine.

# Launching the processes

```
# MPI: 4 processes on this machine
mpiexec -n 4 python ping.py

# PyTorch: 2 nodes x 4 GPUs, on each node
torchrun --nnodes=2 --nproc_per_node=4 \
    --rdzv_backend=c10d \
    --rdzv_endpoint=node0:29500 train.py
```

| torchrun sets | Example |
|---|---|
| RANK | 0 … 7 |
| LOCAL_RANK | 0 … 3 |
| WORLD_SIZE | 8 |
| LOCAL_WORLD_SIZE | 4 |
| MASTER_ADDR, PORT | rendezvous |

**Key idea**

The launcher gives each copy its **identity**; the program reads it, never hard-codes it.

Source: PyTorch docs, torchrun (Elastic Launch) and "PyTorch Distributed Overview". MPI standard (mpiexec).

# Mapping ranks to GPUs

```
import os, torch
import torch.distributed as dist

local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)   # first!
dist.init_process_group(backend="nccl")
rank = dist.get_rank()
p = dist.get_world_size()
x = torch.ones(4, device="cuda")    # my GPU
```

- Without `set_device`, every rank uses **GPU 0**.
- Or: `CUDA_VISIBLE_DEVICES=2` → this process sees one GPU.
- mpi4py: local rank via `comm.Split_type`.

**Watch out**

Four ranks on GPU 0 → OOM or a crawl, with three GPUs idle. Set the device **first**.

Source: PyTorch Distributed Overview; `torch.cuda.set_device`; mpi4py `Comm.Split_type`.

# Check yourself

Two minutes with a neighbour.

- 4 nodes $\times$ 8 GPUs, one process per GPU: world size? Node and local rank of rank 13?
- A script forgets `torch.cuda.set_device(local_rank)`. What do you observe?
- A 13B-parameter model, Adam, mixed precision: at least how many 80 GB GPUs for the state?

Part 2

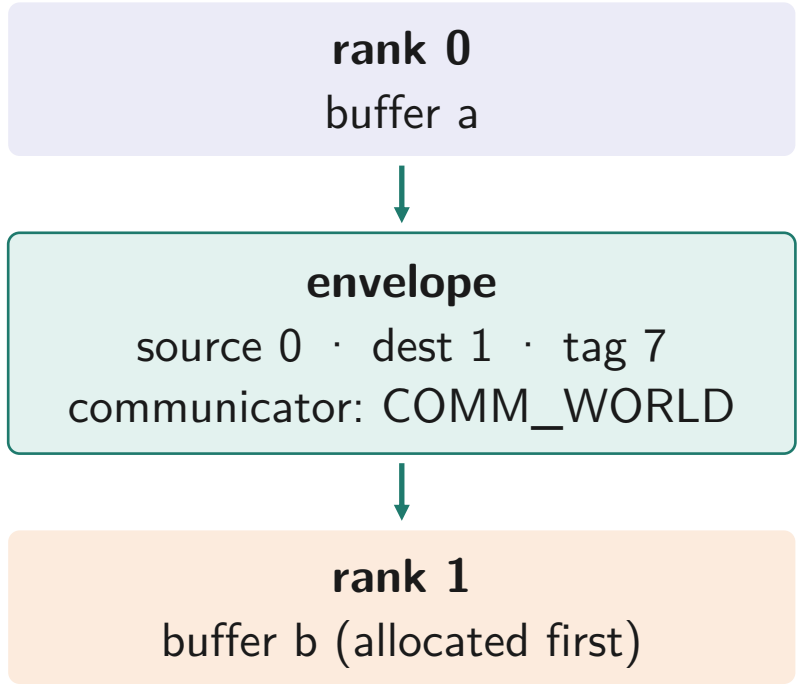
# Point-to-point messages

How do two processes exchange data, and how can they get stuck?

# Send and receive

```
from mpi4py import MPI
import numpy as np
comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    a = np.arange(1000, dtype=np.float64)
    comm.Send(a, dest=1, tag=7)
elif rank == 1:
    b = np.empty(1000, dtype=np.float64)
    comm.Recv(b, source=0, tag=7)
```




**Key idea**

A receive matches by **(communicator, source, tag)**; the bytes land in **your** buffer.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# Buffers versus Python objects

| | `comm.Send(buf, ...)` | `comm.send(obj, ...)` |
|---|---|---|
| Sends | a buffer (NumPy array) | any Python object |
| How | raw bytes, no pickling | pickle $\rightarrow$ bytes $\rightarrow$ unpickle |
| Receive | into your buffer | returns a new object |
| Tiny message, one-way | $\approx$ 1 µs | $\approx$ 8 µs |
| 16 MiB, one-way | $\approx$ 2 ms | $\approx$ 4 ms |

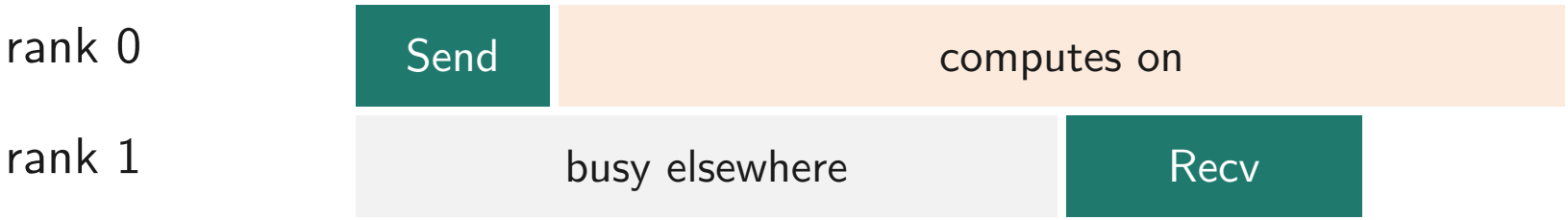
**Key idea**

Numeric data: **uppercase** methods on NumPy arrays. Lowercase is for small Python objects.

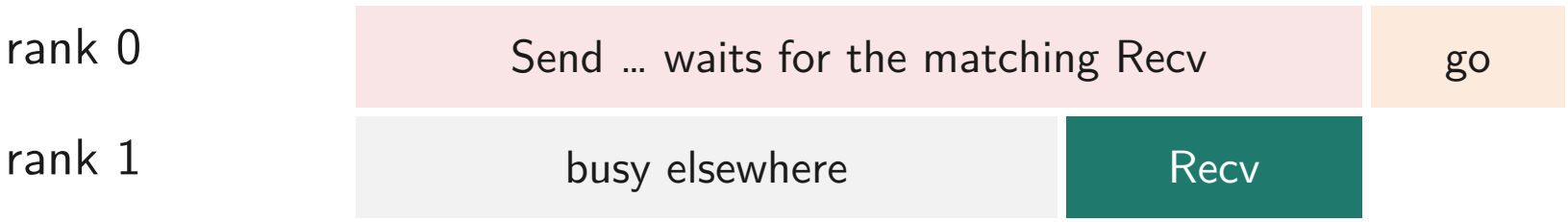
Source: mpi4py documentation (tutorial); measured for these slides: one laptop, 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

# What “blocking” means

**eager**




**rendezvous**




**Watch out**

Send returns when the **buffer is reusable**. Small messages may be buffered; large ones wait for the receiver. Never rely on buffering.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# Deadlock

| rank 0 | rank 1 |
| --- | --- |
| `Send(a, dest=1)` | `Send(a, dest=0)` |
| `Recv(b, source=1)` | `Recv(b, source=0)` |



each Send waits for a Recv

that comes after the other's Send

| Message size | Result (MPICH, laptop) |
| --- | --- |
| 1 KiB | finishes (eager) |
| 64 KiB | **hangs** (rendezvous) |

**Fixes**

1. Parity: even ranks send first, odd ranks receive first.
2. `comm.Sendrecv(a, dest=peer, recvbuf=b, source=peer)`
3. `Isend` + `Irecv`, then wait for both.

Source: measured for these slides: one laptop, 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

# Non-blocking communication

```
peer = 1 - comm.Get_rank()
reqs = [comm.Isend(a, dest=peer, tag=0),
        comm.Irecv(b, source=peer, tag=0)]

independent_work()    # overlaps the transfer

MPI.Request.Waitall(reqs)
use(b)                # now safe to read b
a[:] = 0              # and to overwrite a
```

- **Post** both, then **wait**: no deadlock.
- Do not touch `a` or `b` before the wait.
- Work in between **hides** communication.

**Key idea**

Non-blocking calls separate **starting** a transfer from **finishing** it.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# How messages are matched

```
st = MPI.Status()
comm.Recv(buf, source=MPI.ANY_SOURCE,
          tag=MPI.ANY_TAG, status=st)
print(st.Get_source(), st.Get_tag())
```

- Envelope match: **communicator, source, tag**; wildcards allowed for source and tag.
- **Non-overtaking**: from one sender to one receiver, matching messages arrive **in order**.
- No order is guaranteed **between different senders**.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# Check yourself

- Both ranks call `Send` then `Recv` to each other. When does it work, and is it correct?
- Rank 0 sends tag 1 then tag 2; rank 1 receives tag 2 first. Small and large messages?
- Right after `req = comm.Isend(a, ...)`, may you write into `a`? Why?

Part 3

# Collectives

What exactly does each group operation compute, on every rank?

# What is a collective?

- **Every** rank in the communicator calls it, in the **same order**, with matching sizes.

- Some have a **root**: the rank that provides the data or receives the result.
- Reductions combine elementwise with an associative operation: **sum**, max, min.
- Libraries implement them with **tuned algorithms**, often far faster than a loop of sends.

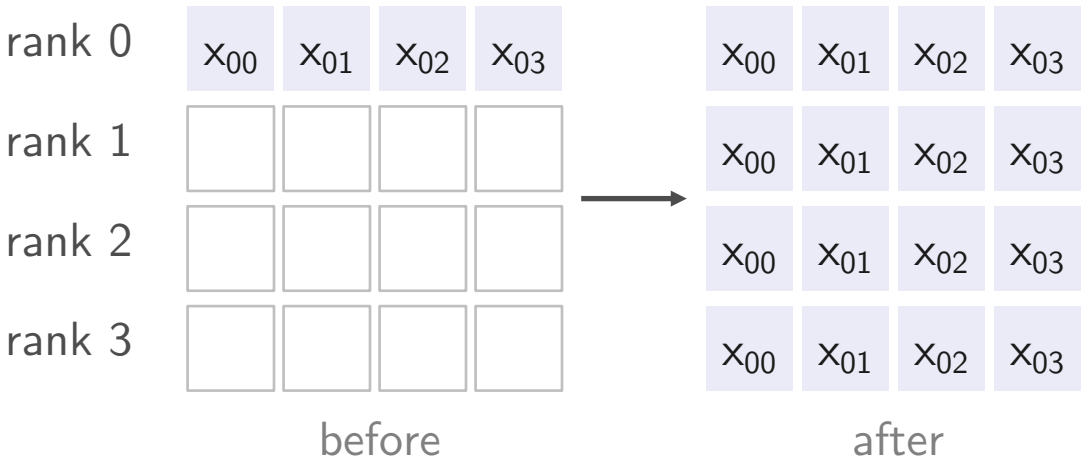
**Watch out**

If one rank skips or reorders a collective, the others wait **forever**.

Source: MPI standard 4.1, Chapter 6; Thakur, Rabenseifner & Gropp, IJHPCA 2005.

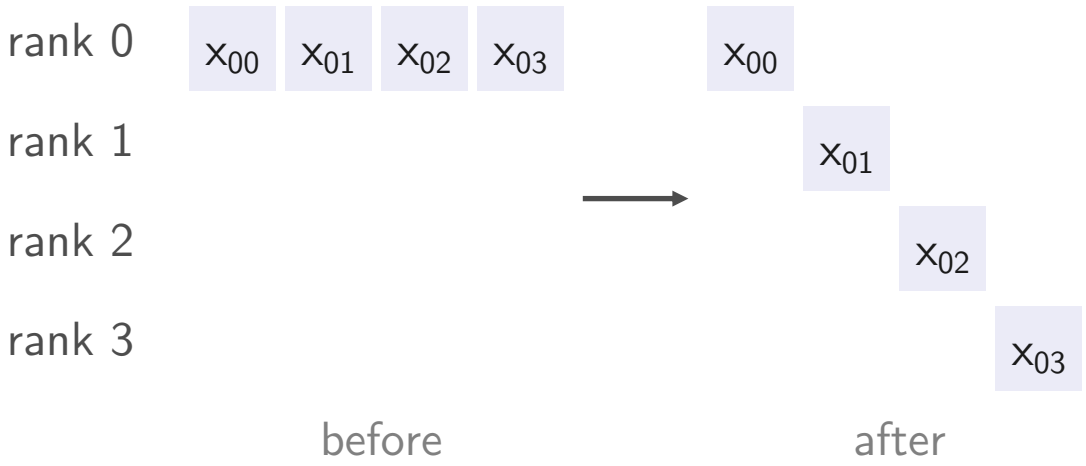
# Broadcast and scatter

## Broadcast (root 0)




```
comm.Bcast(buf, root=0)
```

## Scatter (root 0)




```
comm.Scatter(sendbuf, recvbuf, root=0)
```

**Key idea**

Broadcast: everyone gets **all** of the root's data. Scatter: rank i gets **chunk i**.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

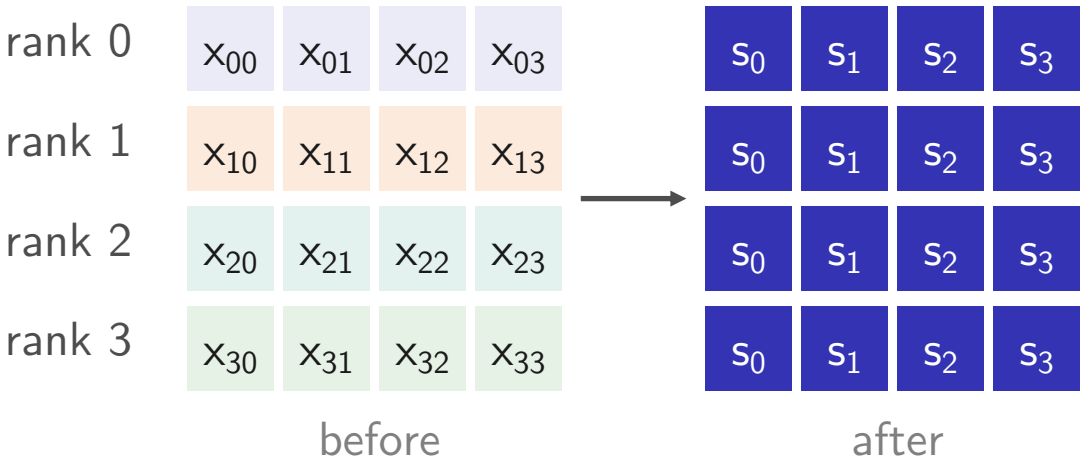
# Reduce and all-reduce

## Reduce, sum (root 0)




```
comm.Reduce(x, out, op=MPI.SUM, root=0)
```

## All-reduce, sum




```
comm.Allreduce(x, out, op=MPI.SUM)
```

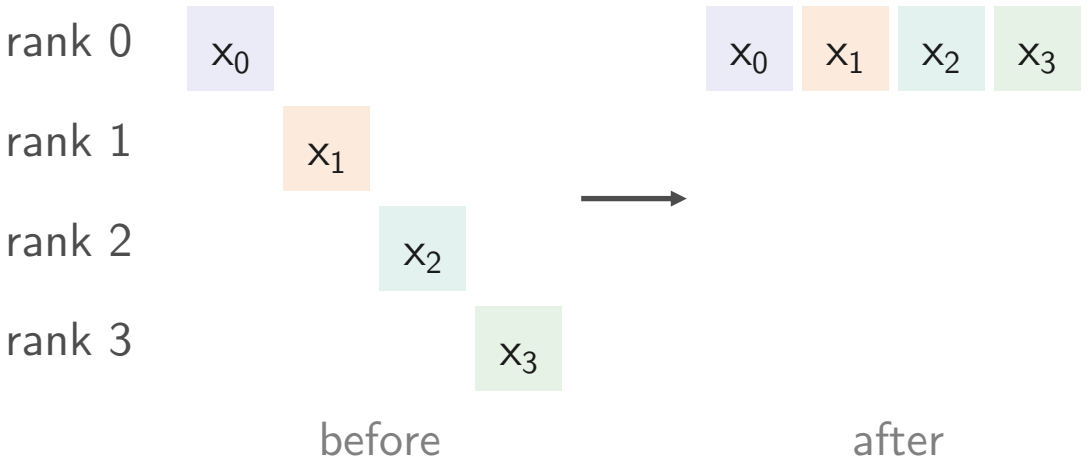
**Key idea**

$s_0 = x_{00} + x_{10} + x_{20} + x_{30}$, elementwise. All-reduce: the sum on **every** rank.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

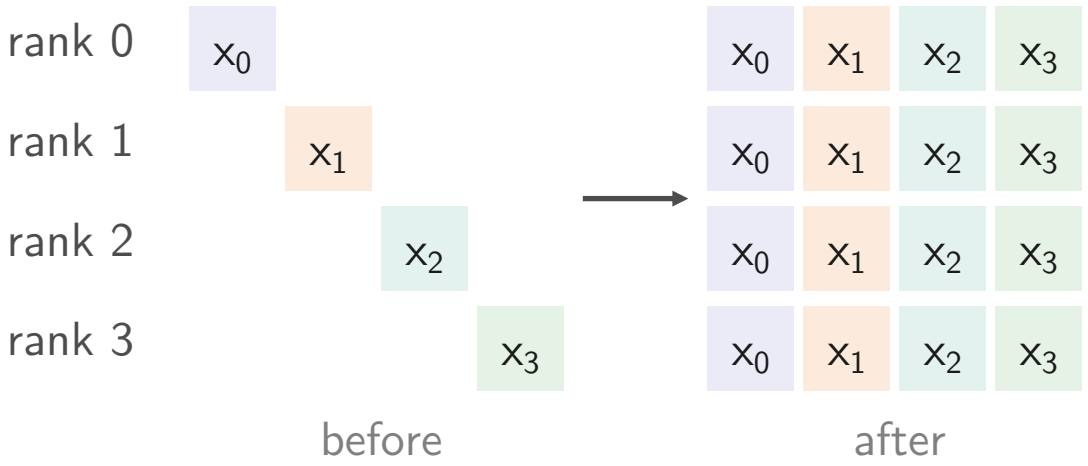
# Gather and all-gather

## Gather (root 0)




```
comm.Gather(x, out, root=0)
```

## All-gather




```
comm.Allgather(x, out)
```

**Key idea**

Gather: the root gets the **concatenation**. All-gather: **every** rank gets it.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# Reduce-scatter

**Reduce-scatter, sum**



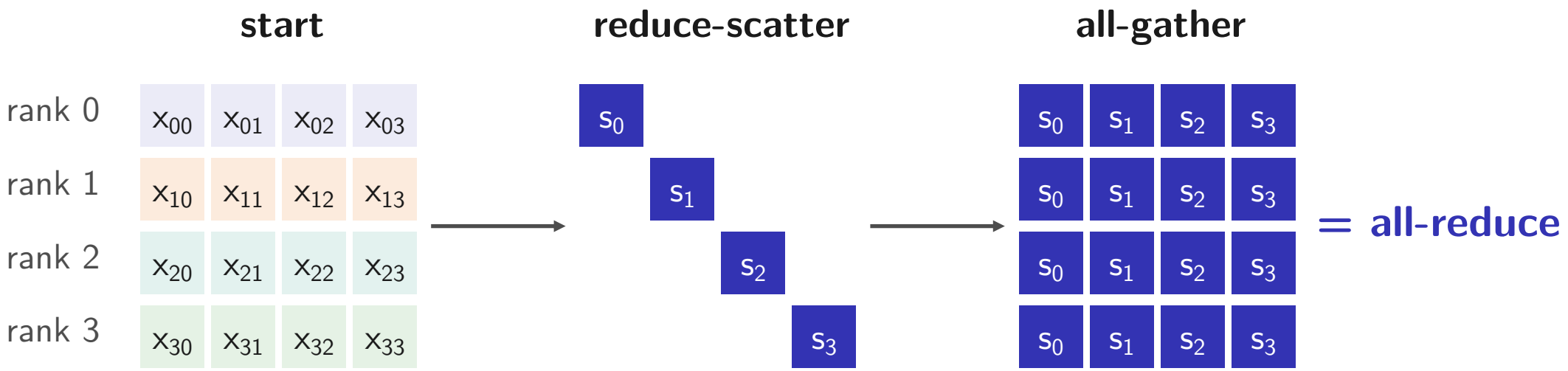

`comm.Reduce_scatter_block(x, out)` (op: sum)

**Key idea**

Rank i receives **chunk i of the sum**: sends n elements, receives n / p.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# All-reduce = reduce-scatter + all-gather




**Key idea**

Each step moves only **n / p** per rank per piece, on **all** links at once — the basis of Rabenseifner's and the ring algorithm.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

# The seven collectives at a glance

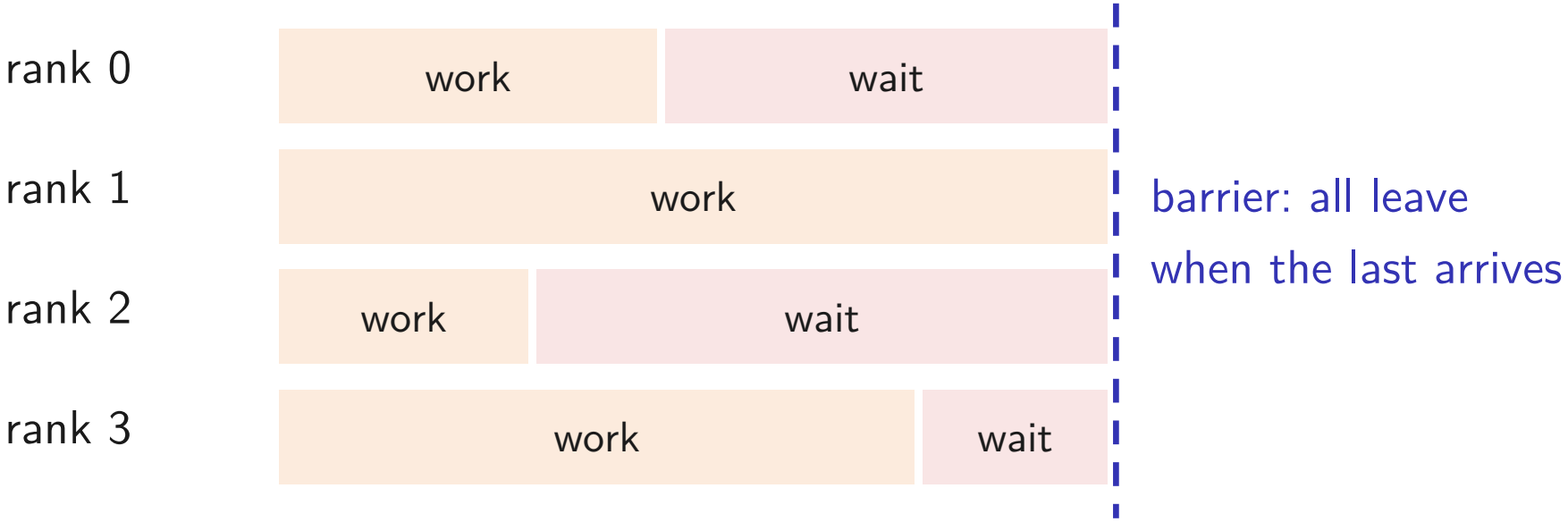
| Collective | In / rank | Out / rank | mpi4py | torch.distributed |
|---|---|---|---|---|
| Broadcast | n (root) | n | `Bcast` | `broadcast` |
| Scatter | n (root) | n / p | `Scatter` | `scatter` |
| Gather | n / p | n (root) | `Gather` | `gather` |
| All-gather | n / p | n | `Allgather` | `all_gather_into_tensor` |
| Reduce | n | n (root) | `Reduce` | `reduce` |
| All-reduce | n | n | `Allreduce` | `all_reduce` |
| Reduce-scatter | n | n / p | `Reduce_scatter_block` | `reduce_scatter_tensor` |

**Key idea**

Sharding pairs: **reduce-scatter** shrinks $n \to n/p$; **all-gather** grows $n/p \to n$.

Source: MPI standard, Chapter 6; mpi4py API reference; torch.distributed documentation.

# Synchronization and barriers




```
comm.Barrier()                  # line everyone up
t0 = MPI.Wtime()
work_to_time()
t = MPI.Wtime() - t0
```

Other collectives **may or may not** synchronize: don't rely on it.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# A first use: averaging gradients

```
from mpi4py import MPI
import numpy as np
comm = MPI.COMM_WORLD
p = comm.Get_size()

g = local_gradient()            # float32 array
comm.Allreduce(MPI.IN_PLACE, g, op=MPI.SUM)
g /= p                          # average

# torch.distributed, same meaning:
# dist.all_reduce(g_t, op=dist.ReduceOp.SUM)
# g_t /= dist.get_world_size()
```

$$(1/p)\ \Sigma\ g \text{ over ranks}$$

$$= \text{gradient of mean loss}$$

**In our pipeline**

T2 builds this all-reduce from **Send/Recv**; T3 builds data-parallel training on it.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

# Check yourself

- Four ranks, each with a 1,000-element vector, call reduce-scatter. What does each receive?
- Write all-reduce as two collectives. How large is each rank's buffer in between?
- Ranks 0, 1, 3 call `Allreduce`; rank 2 calls `Bcast` first. What happens?

Part 4

# The $\alpha$–$\beta$ cost model

Two numbers that predict the time of any message

# The $\alpha$–$\beta$ model

$$T(n) = \alpha + n \,/\, \beta$$

$$n \,/\, T(n) = \beta \cdot n \,/\, (n + \alpha\beta)$$

$$\text{half of } \beta \text{ at } \; n^* = \alpha \cdot \beta$$

| n | n / T(n) |
|---|---|
| 1 KB | 0.91 GB/s |
| 10 KB | 5.0 GB/s |
| 100 KB | 9.1 GB/s |
| 1 MB | 9.9 GB/s |

$\alpha = 1$ µs, $\beta = 10$ GB/s

**Key idea**

Small messages are **latency-bound** ($\alpha$), large ones **bandwidth-bound** ($\beta$); $n^* = \alpha\beta$ separates them.

Source: Hockney, Parallel Computing 20(3), 1994; Thakur, Rabenseifner & Gropp, IJHPCA 2005.

# Fitting $\alpha$ and $\beta$ by least squares

$$\text{minimize} \quad S(\alpha, \gamma) = \Sigma\ (t - \alpha - \gamma \cdot n)^2, \quad \gamma = 1\ /\ \beta$$

$$\partial S/\partial\alpha = 0 \quad \Rightarrow \quad \alpha = \text{mean}(t) - \gamma \cdot \text{mean}(n)$$

$$\partial S/\partial\gamma = 0 \quad \Rightarrow \quad \gamma = \text{Cov}(n, t)\ /\ \text{Var}(n)$$

$$\beta = 1\ /\ \gamma$$

**Key idea**

A linear regression of time on size: **intercept** $= \alpha$, **slope** $= \mathbf{1}/\beta$.

# Log-spaced sizes need relative error

$$\text{minimize} \;\; \Sigma \left( \; (t - \alpha - \gamma \cdot n) \; / \; t \; \right)^2$$

$$= \text{least squares on rows } (1, n) \; / \; t, \;\; \text{target } 1$$

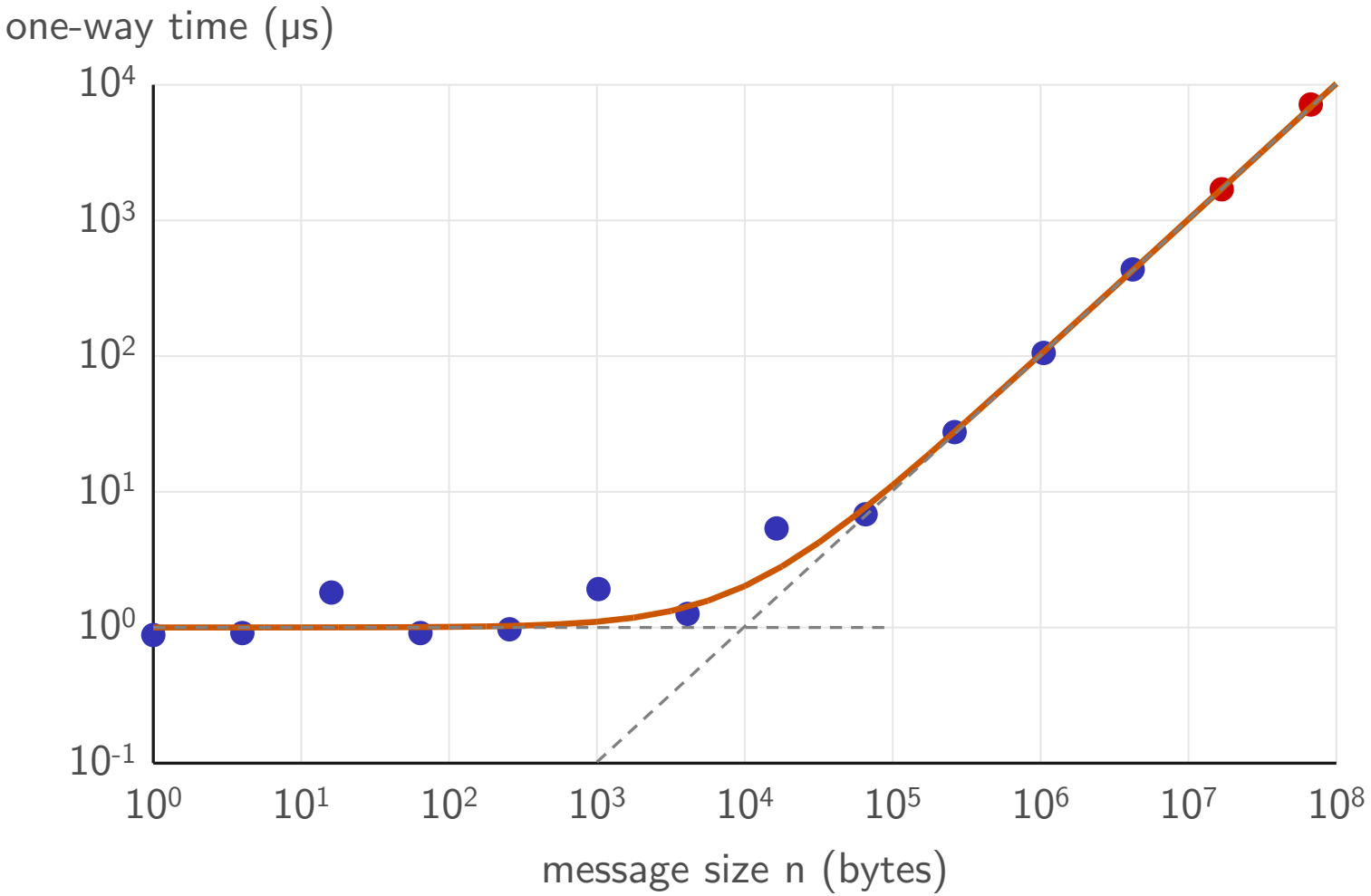
| **Fit to all 14 sizes (1 B – 64 MiB)** | $\alpha$ | $\beta$ |
|---|---|---|
| Plain least squares | **-5.6 µs** | 9.4 GB/s |
| Relative least squares | 1.0 µs | 9.8 GB/s |

**Watch out**

Plain least squares lets the big messages' noise set $\alpha$ — here it is **negative**. Weight by $1/t^2$ so every size counts.

Source: measured for these slides: one laptop, 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

# A real fit, and a prediction

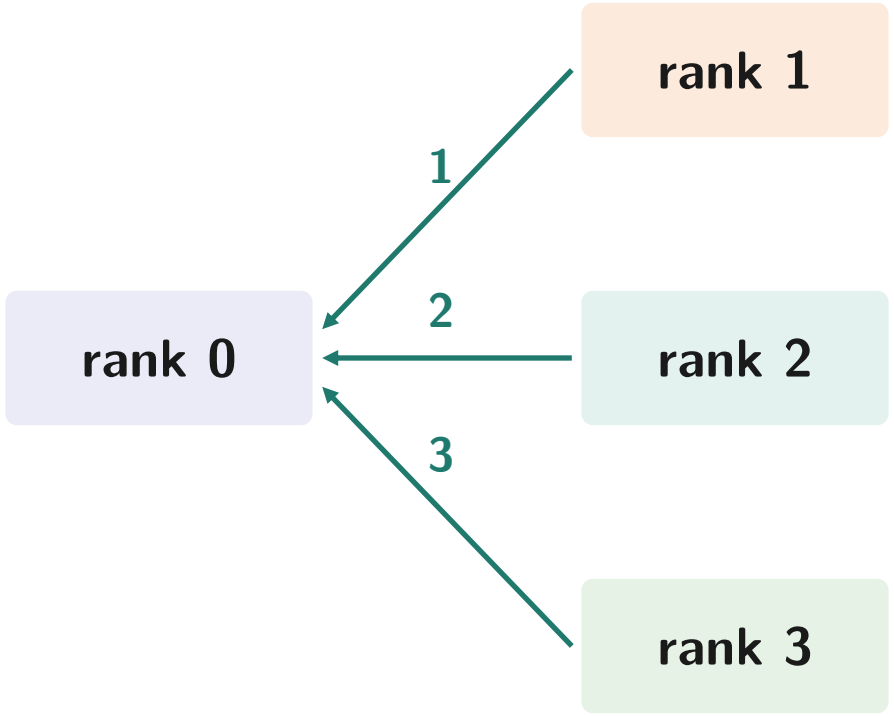



| **Fit ($\leq$ 4 MiB)** | |
|---|---|
| $\alpha$ | 1.0 µs |
| $\beta$ | 9.8 GB/s |
| 64 MiB predicted | 6.8 ms |
| 64 MiB measured | 7.2 ms |

blue: fitted points · red: held out

Source: measured for these slides: one laptop, 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

# A naive all-reduce




p − 1 sends in, one after another; then p − 1 sends back

$$T = 2\ (p-1)(\alpha + n\ /\ \beta)$$

$$n = 2\ \text{GB},\ p = 8,\ \beta = 450\ \text{GB/s}$$

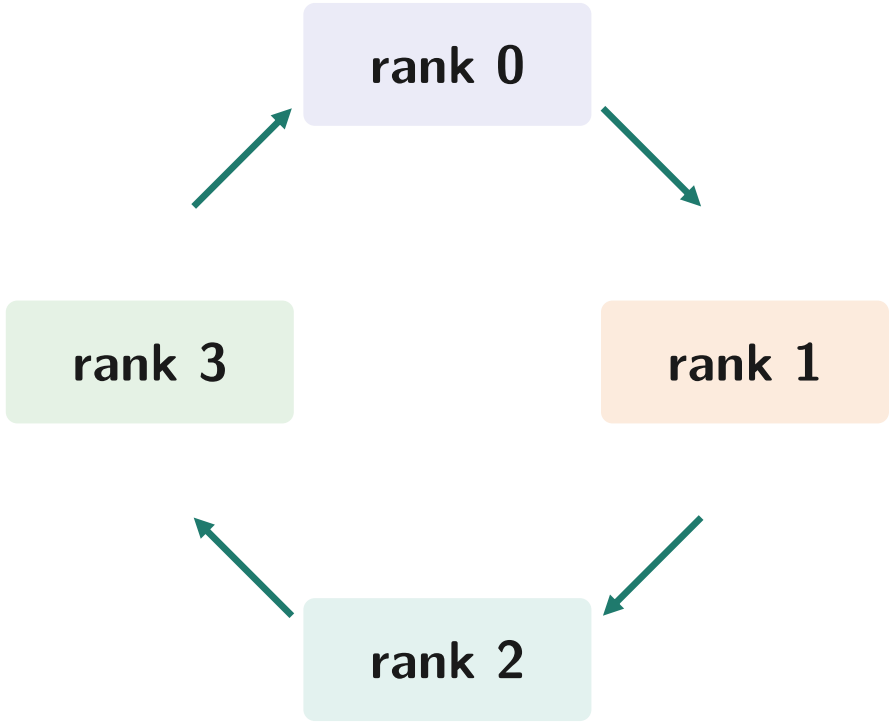
$$T \approx 62\ \text{ms}$$

**Watch out**

Rank 0's link carries **everything**; time grows with p.

Source: H100 datasheet: NVLink 900 GB/s bidirectional → 450 GB/s per direction; idealized.

# Preview: the ring all-reduce




each step: every rank sends n/p to its neighbour, all at once

$$T = 2\,(p-1)(\alpha + n\,/\,(p\,\beta))$$

$$\text{bandwidth term} \to 2n\,/\,\beta \quad (\text{no } p)$$

| **n = 2 GB, p = 8** | **Time** |
|---|---|
| Naive (via rank 0) | $\approx$ 62 ms |
| Ring | $\approx$ 7.8 ms |

**Check**

Same answer, **p× less time**: use every link at once. Details next lecture.

Source: Thakur, Rabenseifner & Gropp, IJHPCA 2005; NVLink from the H100 datasheet; idealized.

# Check yourself

- $\alpha = 5$ µs, $\beta = 25$ GB/s. Predict T for 1 KB and 100 MB. What is n*?
- Your fitted $\alpha$ is negative. What went wrong, and how do you fix the fit?
- Going from 8 to 16 GPUs: how does each all-reduce's bandwidth term change?

IN-CLASS BUILD

# Ping-pong and an $\alpha$–$\beta$ fit

Pairs: measure, fit, then predict a message you have not measured.

# Exercise 1: ping-pong

```
from mpi4py import MPI
import numpy as np
comm = MPI.COMM_WORLD
rank, reps = comm.Get_rank(), 100
for k in range(0, 25, 2):          # 1 B ... 16 MiB
    buf = np.zeros(2**k, dtype=np.uint8)
    comm.Barrier(); t0 = MPI.Wtime()
    for _ in range(reps):
        if rank == 0: comm.Send(buf, dest=1)
        comm.Recv(buf, source=1 - rank)
        if rank == 1: comm.Send(buf, dest=0)
    if rank == 0:
        print(2**k, (MPI.Wtime() - t0) / reps)
```

**DO**

1. Predict T(1 B), T(16 MiB).
2. Run: `mpiexec -n 2`
3. Repeat: `send` / `recv`.

**ASK**

Why can this loop never deadlock?

Source: mpi4py documentation; code run on one laptop (MPICH 5.0.2, mpi4py 4.1.2).

# Exercise 2: fit, then predict

```
import numpy as np
n, rtt = np.loadtxt("ping.txt", unpack=True)
t = rtt / 2                          # one-way time
A = np.column_stack([np.ones_like(n), n])
w = 1 / t                            # relative error
c, *_ = np.linalg.lstsq(A * w[:, None], t * w,
                        rcond=None)
alpha, beta = c[0], 1 / c[1]
print(f"alpha = {alpha*1e6:.2f} us")
print(f"beta  = {beta/1e9:.2f} GB/s")
n_big = 2**26                        # predict 64 MiB
print("T =", alpha + n_big / beta, "s")
```

**DO**

Write the 64 MiB prediction **before** measuring it. Then measure: how far off?

**STRETCH**

Fit with plain least squares too. Which $\alpha$ is believable?

Source: NumPy documentation, `numpy.linalg.lstsq`; code run on the laptop data of Part 4.

# Exercise: what to expect

| Quantity (laptop, shared memory) | Value |
|---|---|
| one-way, 1 B | $\approx$ 0.9 µs |
| one-way, 16 MiB | $\approx$ 1.7 ms |
| fit: $\alpha$, $\beta$ | $\approx$ 1.0 µs, $\approx$ 9.8 GB/s |
| 64 MiB: predicted vs measured | 6.8 ms vs 7.2 ms |
| pickled `send`, tiny message | $\approx 8\times$ slower than `Send` |

**In our pipeline**

The model predicted an unmeasured size within $\approx 5\%$. T2 repeats this for NCCL on GPUs and for **two servers**.

Source: measured for these slides: one laptop, 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

# Key takeaways

- Large models need **many processes**: 16 bytes per parameter exceeds one GPU.
- SPMD: one program, p processes; **rank** picks the work, **local rank** picks the GPU.
- Send returns when the buffer is reusable; mismatched blocking order can **deadlock**.
- Seven collectives with exact semantics; **all-reduce = reduce-scatter + all-gather**.
- $\mathbf{T(n) = \alpha + n/\beta}$; fit with **relative** least squares, then predict before measuring.
- Naive all-reduce costs $\propto$ p; the ring's bandwidth term is **independent of p**.

# Next time, and readings

| When | What |
|---|---|
| Friday (this week) | T1 due |
| After this lecture | T2 and Lab 4 released |
| Week 11, Lecture 1 | NCCL and GPU communication: ring all-reduce, torch.distributed |
| Week 11, Lecture 2 | Multi-server communication |

**READINGS**

Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 2005.

PyTorch documentation: "PyTorch Distributed Overview".